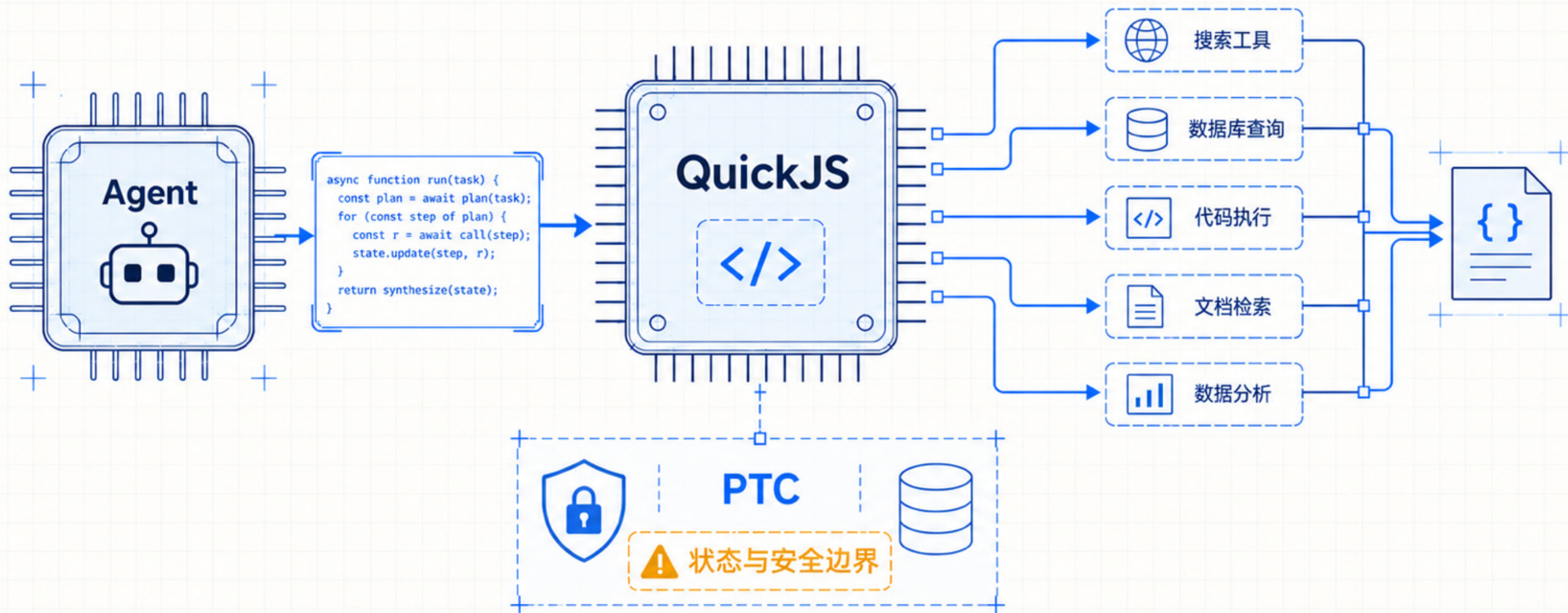


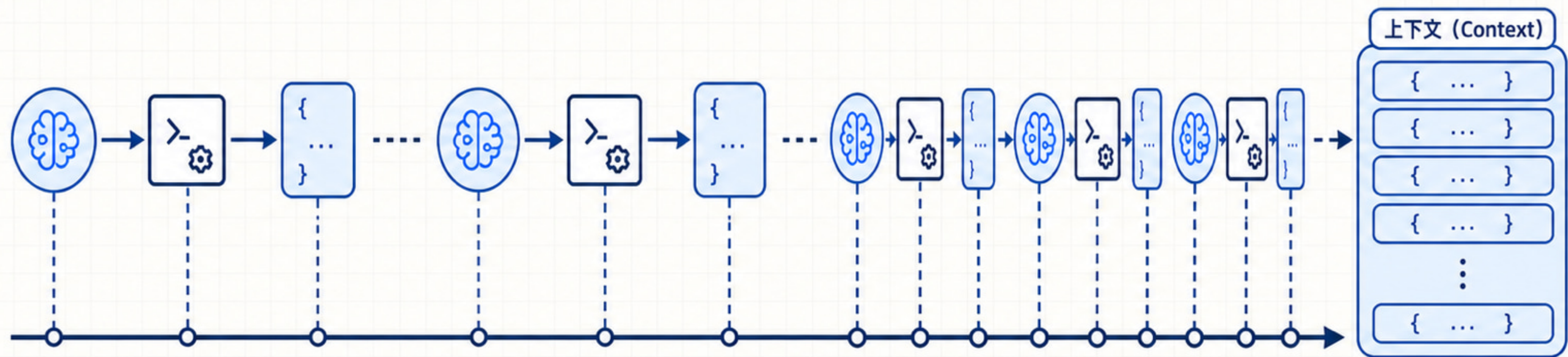
Deep Agents 实战

第 19 讲：Interpreters — 让 Agent 用代码编排工具与数据



顺序工具调用的问题，不在工具本身

中间结果越多，模型上下文越拥挤



⚠ 重复 × 80



模型决定



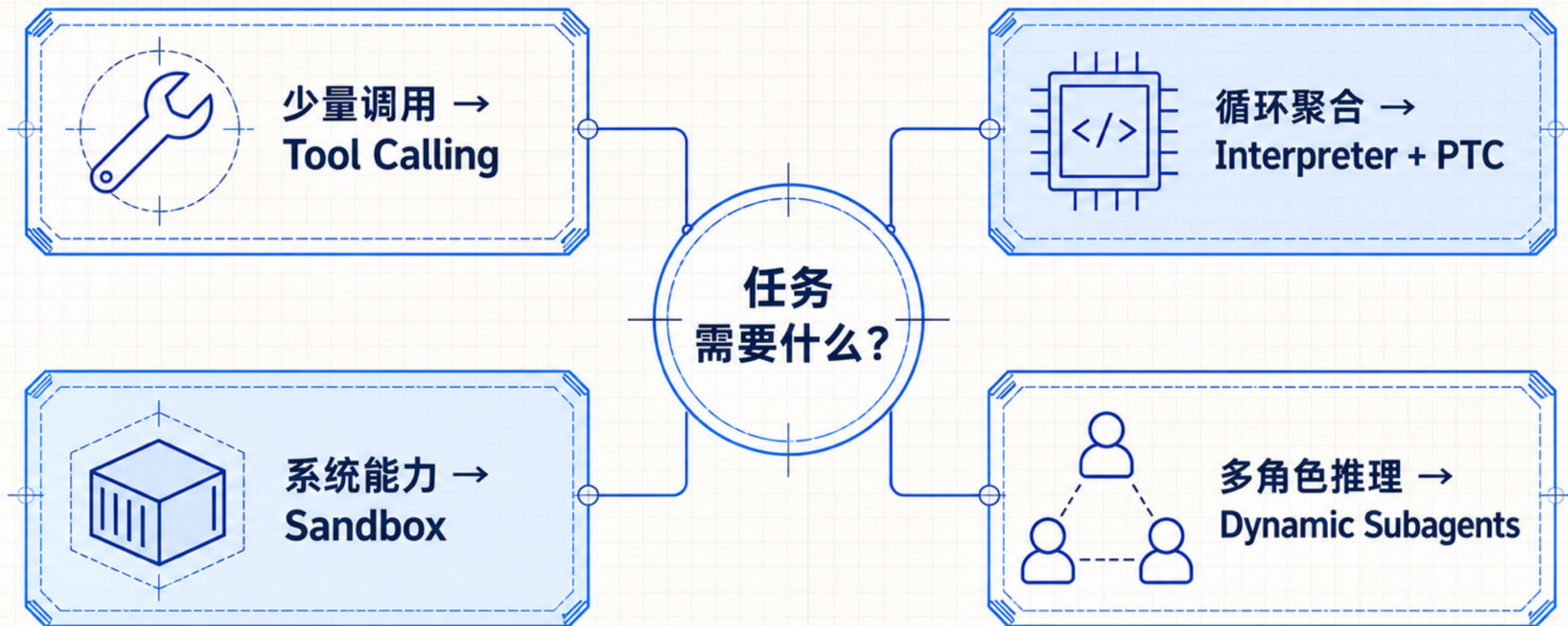
工具返回



结果进入上下文

先看任务形状，再决定执行机制

调用规模、系统能力和推理角色是三个分岔点



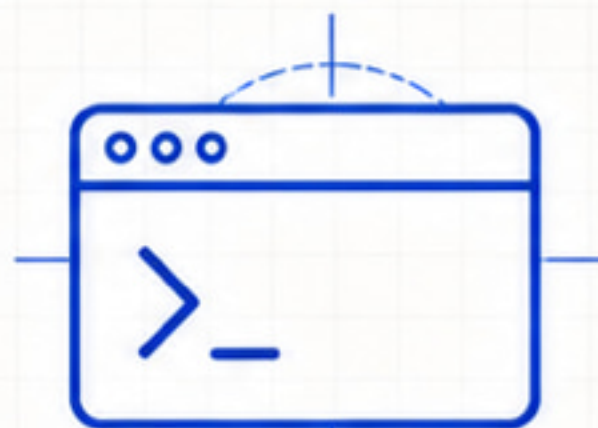
QuickJS 是引擎，langchain-quickjs 是 Python 桥

CodeInterpreterMiddleware 才是 Deep Agents 的接入点



三步把 Interpreter 接进 Agent

1 安装依赖



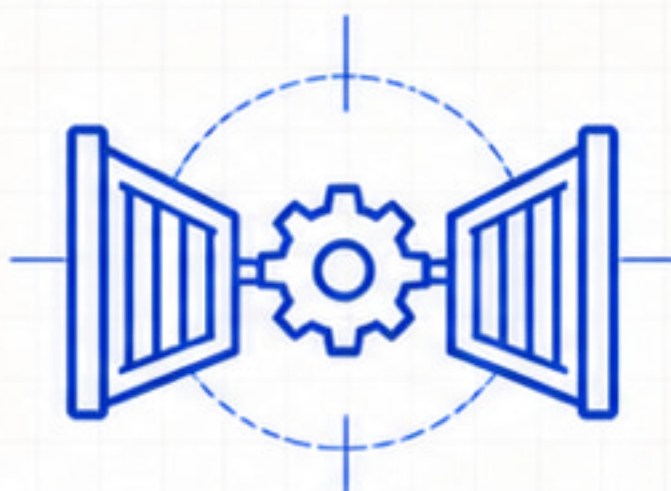
```
uv add "deepagents[quickjs]"
```

2 导入中间件



```
from langchain_quickjs import CodeInterpreterMiddleware
```

3 加入 middleware



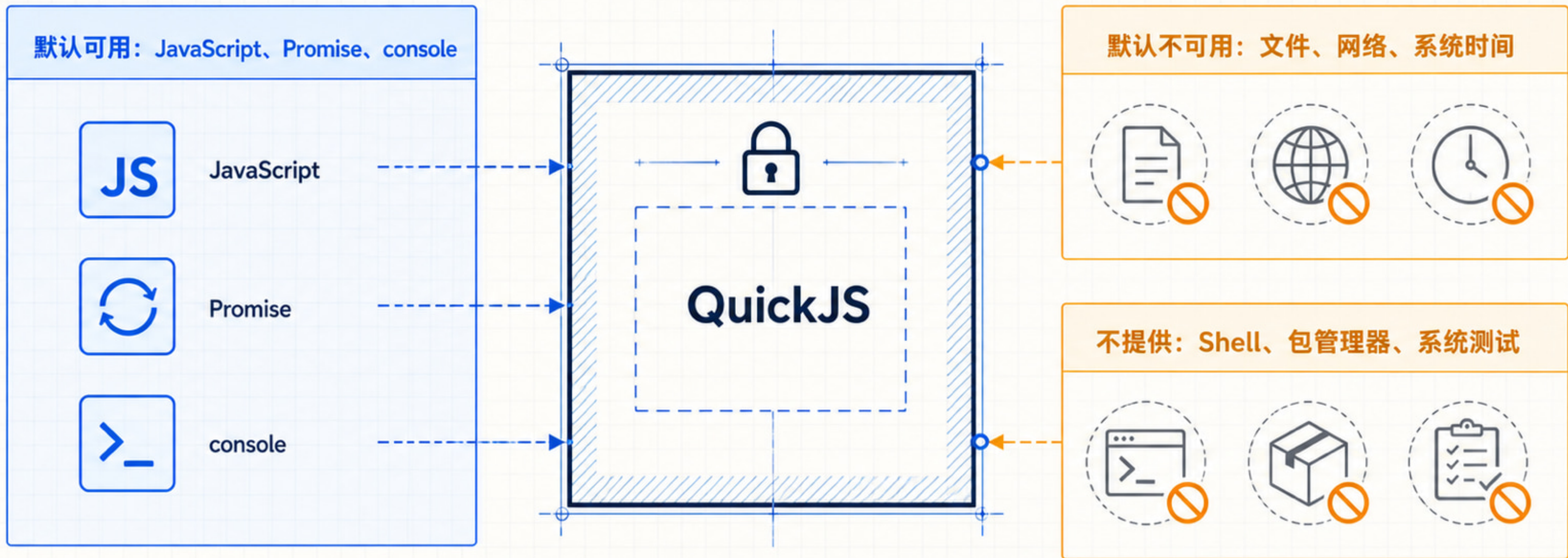
```
middleware=[CodeInterpreterMiddleware()]
```

```
await agent.ainvoke(...)
```

★ 推荐异步调用

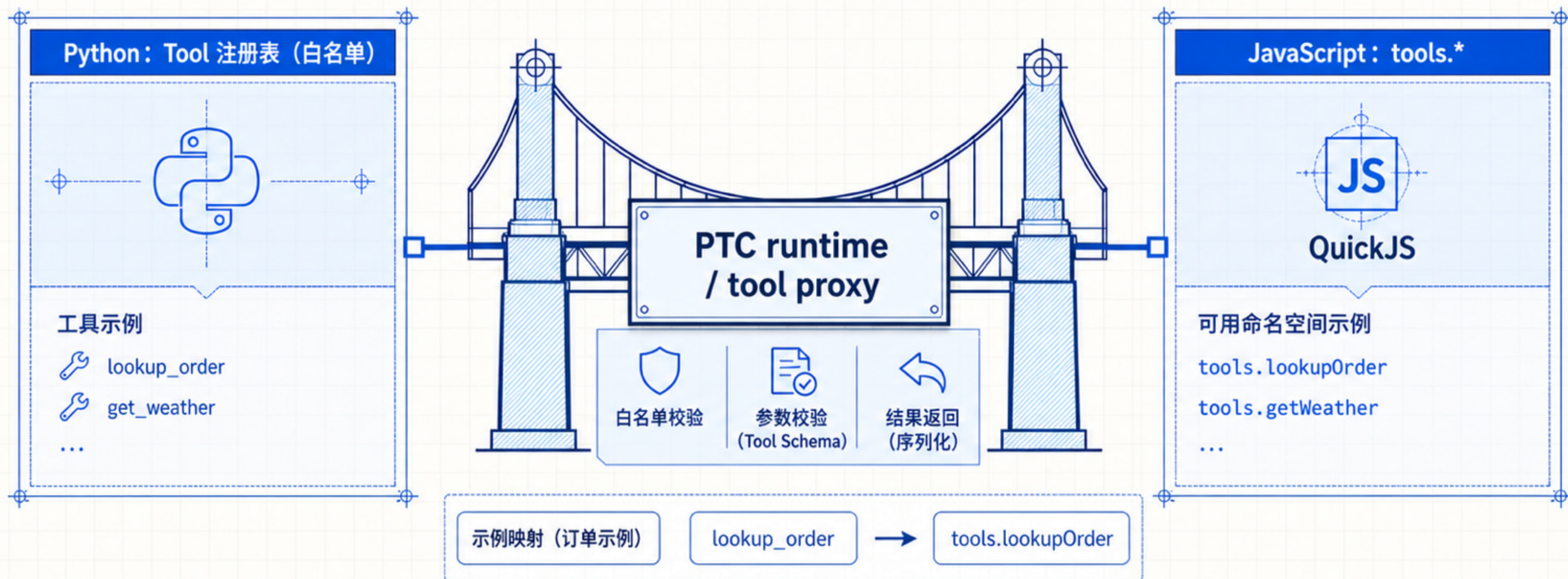
默认封闭，权限通过工具逐项打开

QuickJS 不是轻量版 Shell



PTC runtime 把白名单工具映射为 tools.*

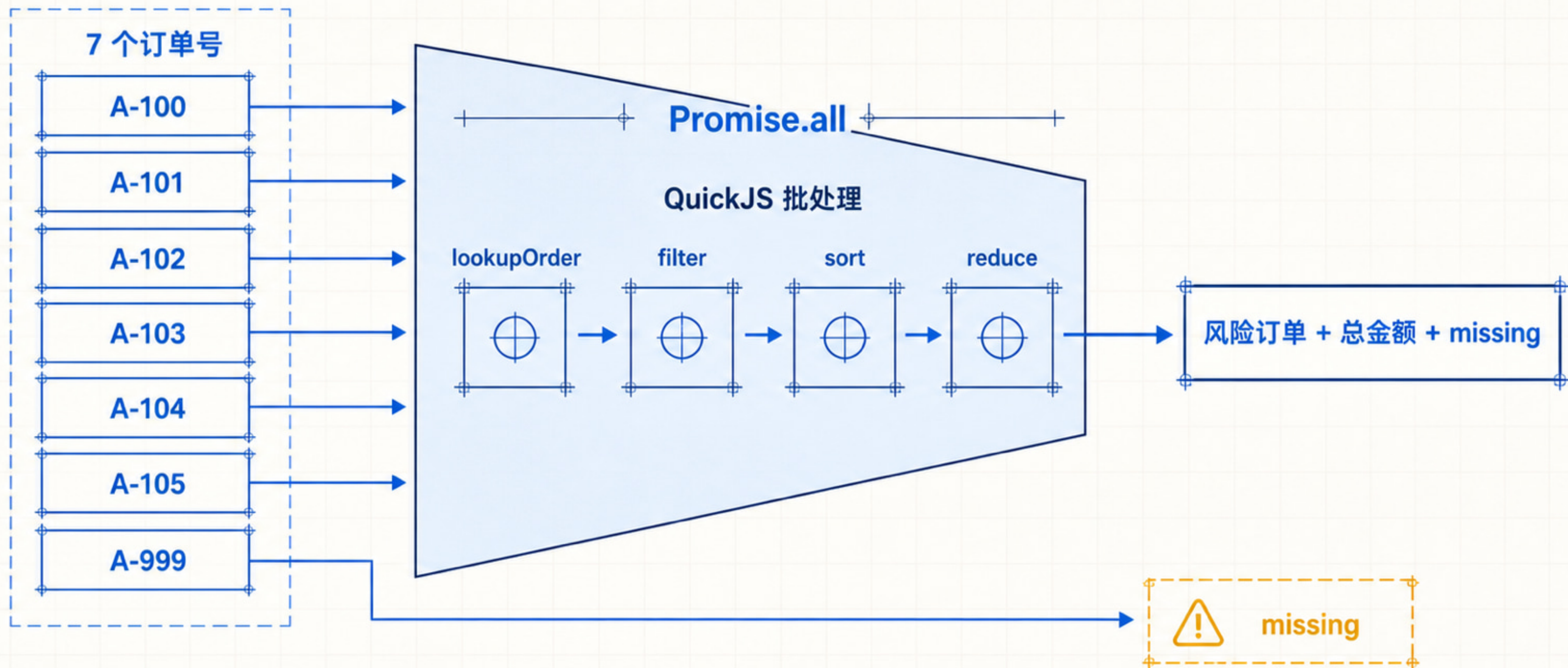
lookupOrder 只是订单示例，不是专用桥接函数



函数名转 camelCase, 参数仍遵循 Tool Schema

并行查询留在代码，模型只收汇总结果

Promise.all 覆盖全部输入，缺失项单独归类



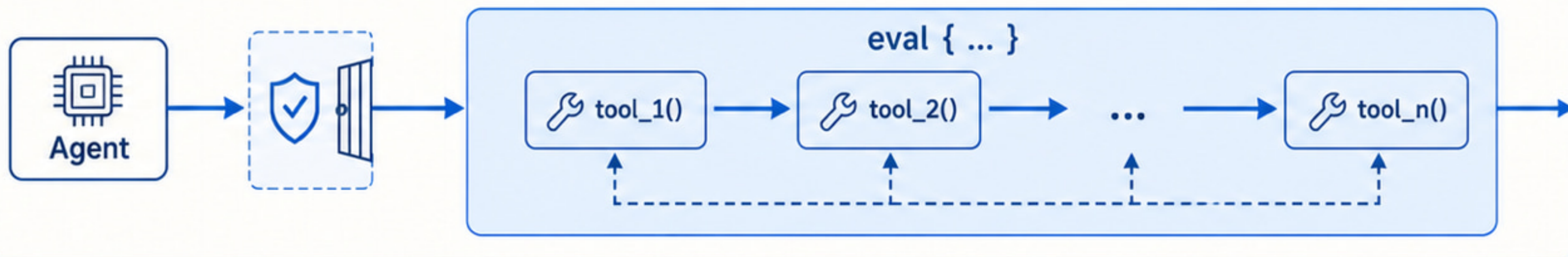
PTC 改变调用路径，也改变审批边界

interrupt_on 不会逐次覆盖 PTC 内部调用

普通 Tool Calling: 逐次审批



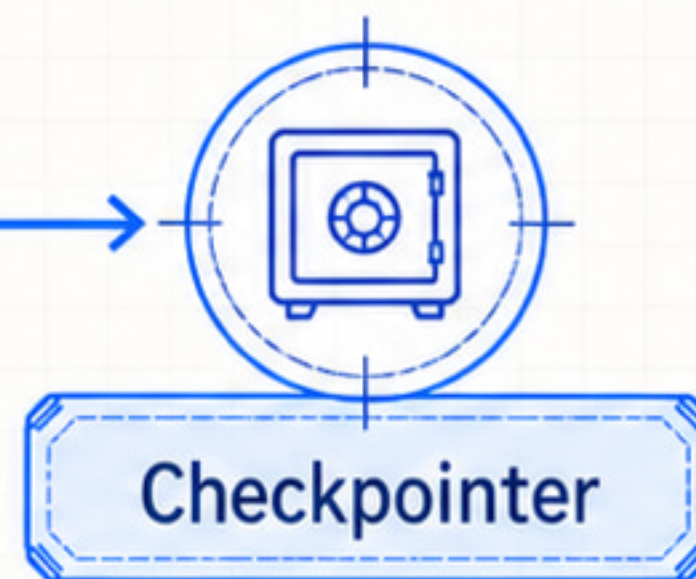
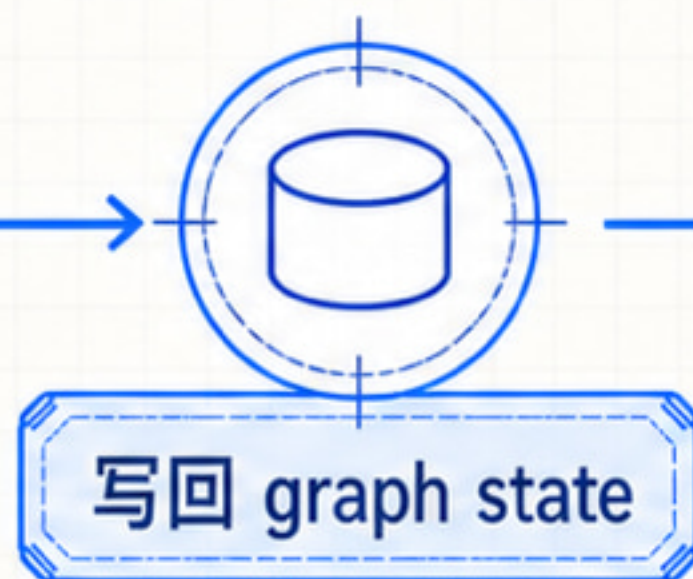
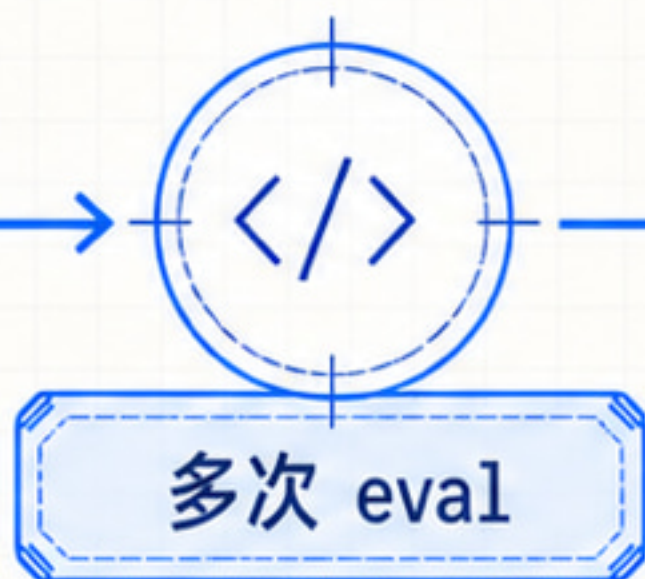
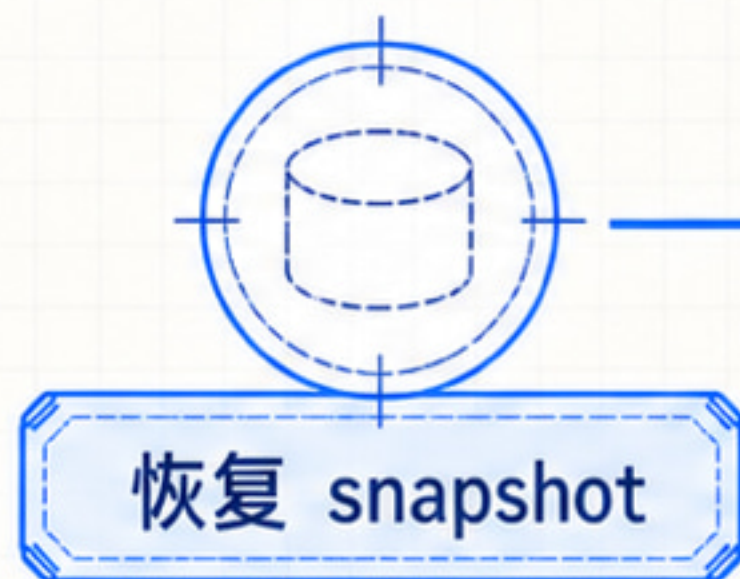
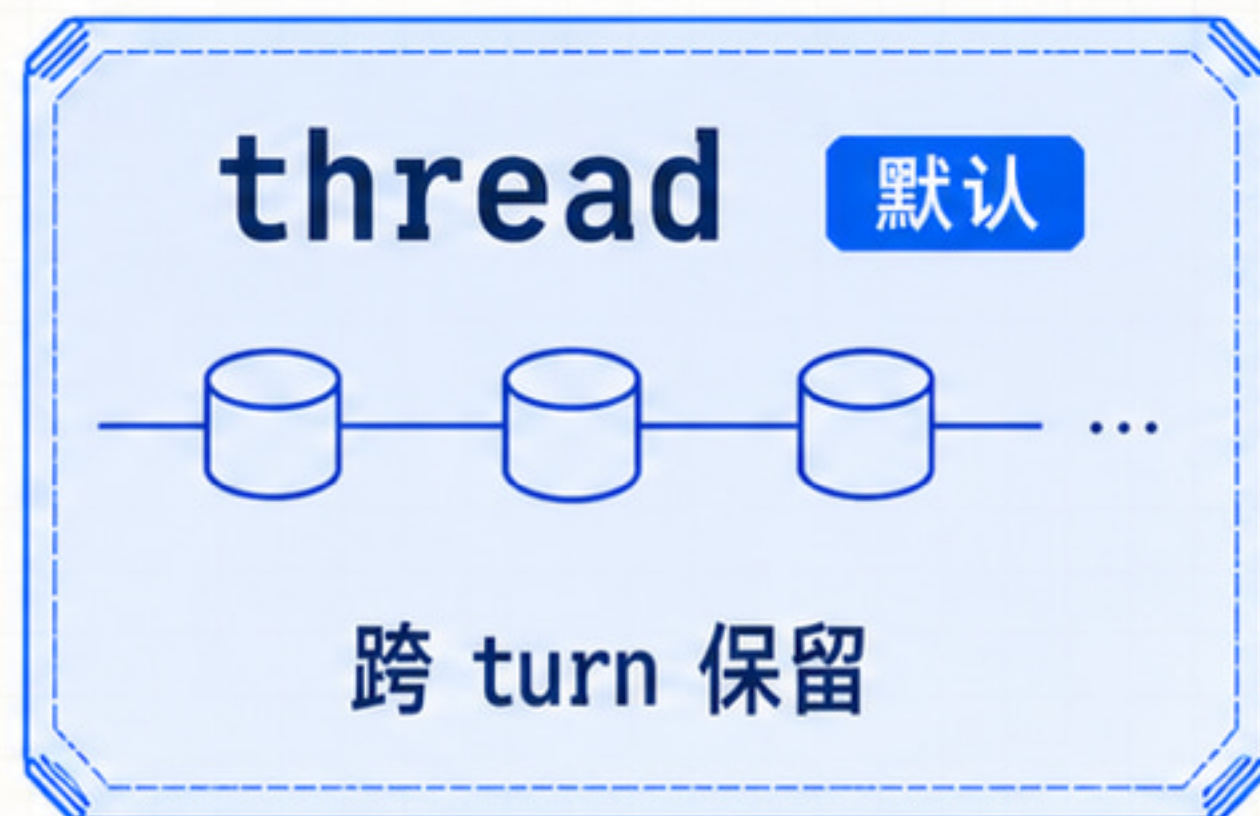
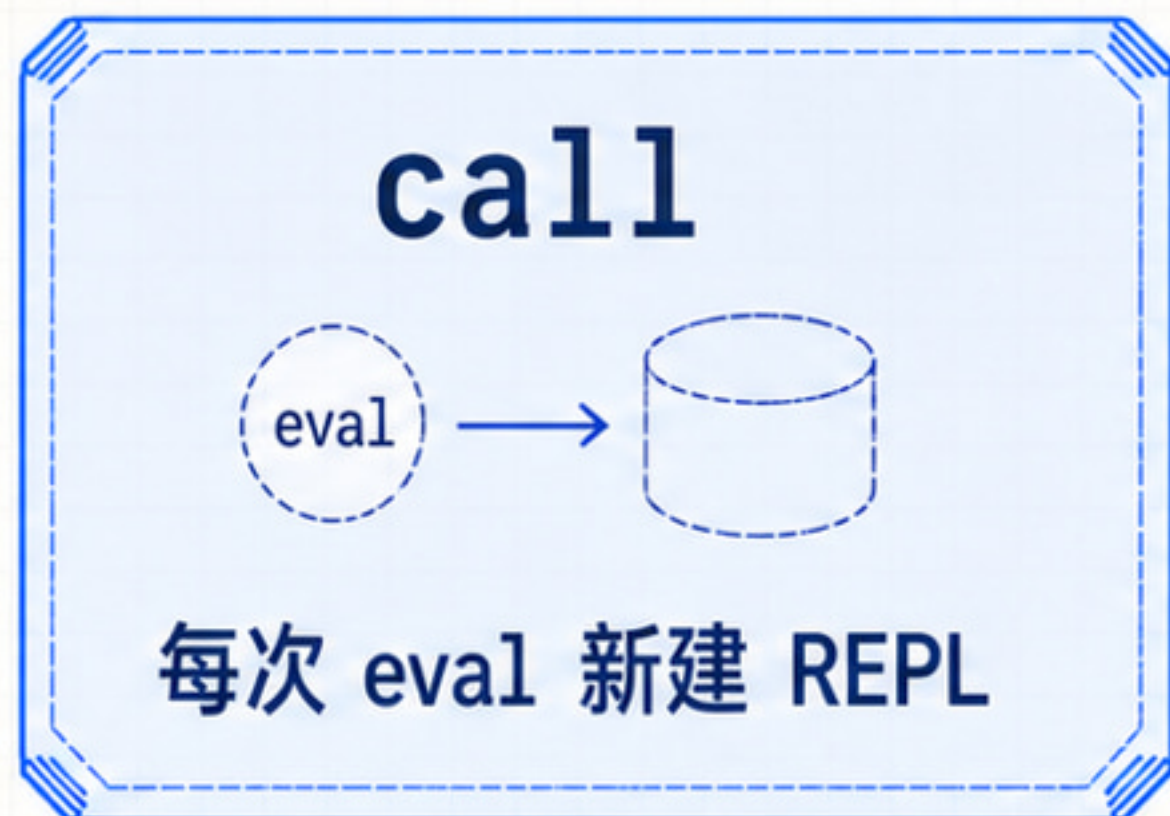
PTC: eval 内部批量执行



⚠
高副作用写操作
保留普通路径

mode 决定状态保留到哪里

Checkpoint 只在 durable thread 或 time travel 时需要



⚠ Snapshot 不会撤销 PTC 工具副作用

安全配置先收紧三件事：能力、资源、状态

PTC 白名单决定能做什么，预算和 mode 决定能有多少、留多久

能力

ptc=["web_search"]
subagents=False

只开放必需入口

资源

memory_limit=64 * 1024 * 1024
timeout=5.0
max_ptc_calls=16

按合法批次收紧

返回与状态

max_result_chars=4000
mode="call"
capture_console=False

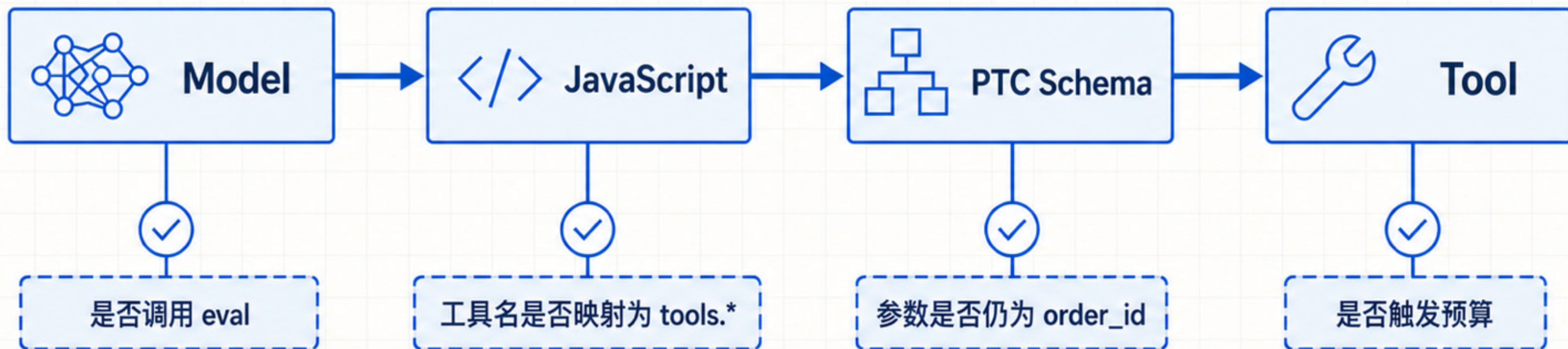
减少返回与跨调用残留

```
CodeInterpreterMiddleware(ptc=["web_search"], subagents=False, timeout=5.0, max_ptc_calls=16, mode="call")
```

示例起点：按真实工作负载调整

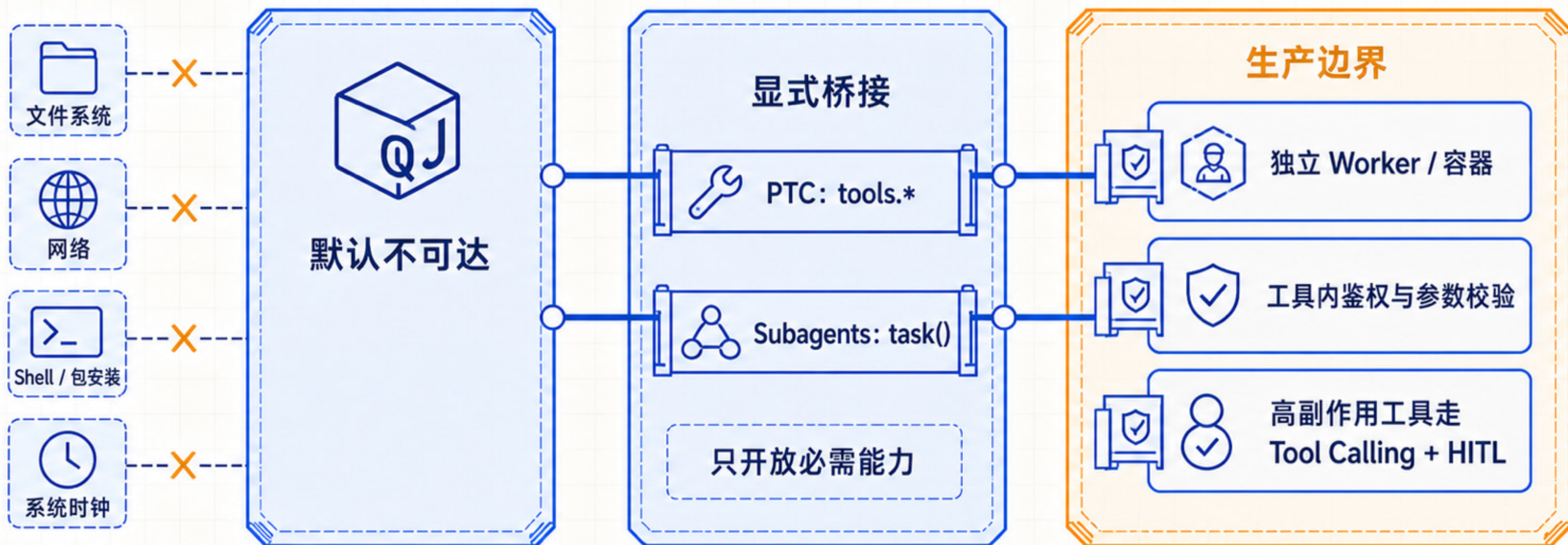
沿调用链排错，别只改 Prompt

Model → JavaScript → PTC Schema → Tool



QuickJS 隔离的是能力，不是宿主进程

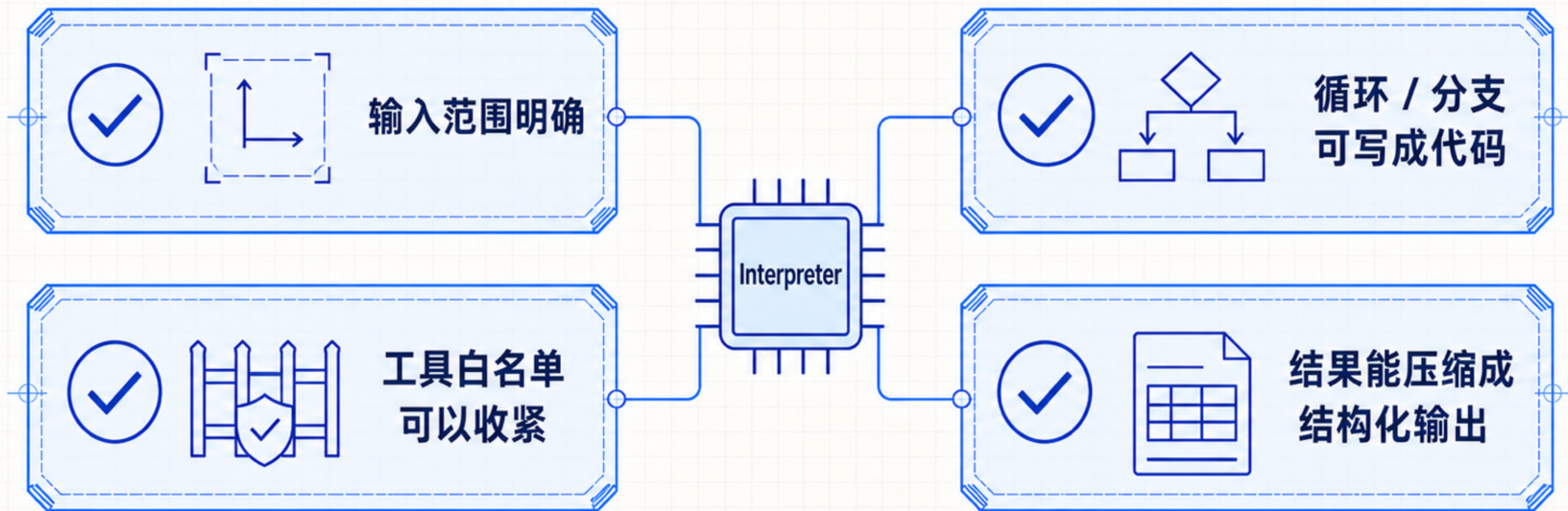
解释器与 Python 同进程运行；它不是 VM、容器或 Sandbox backend



PTC 工具能做什么，Interpreter 代码就能做什么

满足这四个条件，再用 Interpreter

任务能写成短程序，边界也能被明确限制



模型定目标，代码跑流程，工具守权限